



密码学引论实验报告

姓名: 江李阳, 王健一, 邱泽辉

学号: 202400460042, 202400460037, 202400460046

专业: 密码科学与技术

日期: June 3, 2026

Contents

1	实验目的	2
2	实验环境	2
3	实验原理	2
3.1	Diffie-Hellman 密钥协商协议	2
3.2	认证密钥协商协议	2
3.2.1	端到端密钥协商	2
3.3	传输层安全协议标准	3
4	项目架构	4
4.1	安全目标	4
4.2	流程设计	5

1 实验目的

针对 AI 智能体协作航班预定场景, 结合密钥建立协议、电子支付协议等技术, 设计 AI 智能体协作安全协议。

2 实验环境

- 硬件环境: 13th Gen Intel(R) Core(TM) i5-13420H (12 CPUs), 2.1GHz 处理器;16384MB RAM 内存。
- 软件环境: Visual studio code
- 操作系统: Arch Linux(kernel:x86_64 Linux 6.6.87.2-microsoft-standard-WSL2)
- 编程语言: Python

3 实验原理

3.1 Diffie-Hellman 密钥协商协议

Diffie-Hellman 密钥协商协议基于离散对数问题困难性 (DLP), 实现通信双方 P_A, P_B 共享相同会话密钥, 协议基本过程如下:

1. P_A 随机选择 $r_A \in [1, q-1]$, 计算 $R_A = g^{r_A}$, 并将 R_A 发送给 P_B ;
2. P_B 随机选择 $r_B \in [1, q-1]$, 计算 $R_B = g^{r_B}$, $K_{AB} = R_A^{r_B}$, 并将 R_B 发送给 P_A ;
3. P_A 计算 $K_{AB} = R_B^{r_A}$.

以上过程实现了 P_A, P_B 共同协商建立了会话密钥 K_{AB} :

$$K_{AB} = R_B^{r_A} = (g^{r_B})^{r_A} = g^{r_B r_A} = (g^{r_A})^{r_B} = R_A^{r_B}$$

但是该密钥协商协议难以抵御中间人攻击, 根本原因在于基础版的 Diffie-Hellman 密钥交换协议缺乏认证, 合法用户无法确认所收到消息来源的真实性。

3.2 认证密钥协商协议

认证密钥协商 (authenticated key exchange, AKE) 允许两个或多个用户在不安全网络信道上进行身份和消息认证, 并协商会话密钥, 从而建立安全信道进行通信。

3.2.1 端到端密钥协商

端到端密钥协商 (station to station, STS) 是 AKE 协议中的一种, 其本质为一种带有认证功能的 Diffie-Hellman 协议, 使用“认证”的公钥数字签名提供认证性。其认证即认证公钥的真实性, 是通过可信第三方: 证书认证机构 (certification authority, CA) 进行的。用户在

CA 处注册并得到证书 (certificate), 该证书实质是 CA 对用户身份与公钥的数字签名, 其作用是验证用户公钥的合法性, 如 CA 为用户 P_A 颁发的证书如下:

$$Cert(P_A) = (ID_A, pk_A, \sigma_{CA,A})$$

其中 ID_A 为用户 P_A 的唯一标识符, pk_A 为用户 P_A 的公钥, $\sigma_{CA,A}$ 为 CA 用自己的私钥 sk_{CA} 对消息 $ID_A||pk_A$ 的数字签名。STS 协议基本过程如下:

0. 首先通信双方 P_A, P_B 需要在 CA 处注册得到各自的证书:

$$Cert(P_A) = (ID_A, pk_A, \sigma_{CA,A}), Cert(P_B) = (ID_B, pk_B, \sigma_{CB,B})$$

1. P_A 随机选择 $r_A \in [1, q-1]$, 然后计算 $R_A = g^{r_A}$, 并将 $Cert(P_A), R_A$ 发送给 P_B .
2. P_B 随机选择 $r_B \in [1, q-1]$, 然后计算 $R_B = g^{r_B}$, $K_{AB} = (R_A)^{r_B}$ 和 $\sigma_B = Sign_{sk_B}(ID_A||R_B||R_A)$, 并将 $Cert(P_B), R_B, \sigma_B$ 发送给 P_A .
3. P_A 首先利用 $Cert(P_B)$ 验证 pk_B 的合法性, 若无效则拒绝通信并退出; 使用 pk_B 验证签名 σ_B , 若无效则拒绝通信并退出; 否则接收, 计算 $K_{AB} = (R_B)^{r_A}$ 和 $\sigma_A = Sign_{sk_A}(ID_B||R_A||R_B)$, 并将 σ_A 发送给 P_B .
4. P_B 首先使用 $Cert(P_A)$ 验证 pk_A 的合法性, 若无效则拒绝通信并退出; 使用 pk_A 验证签名 σ_A , 若无效则拒绝通信并退出; 否则接收, 计算 $K_{AB} = (R_A)^{r_B}$.

STS 协议包含 Diffie-Hellman 协议的主要步骤, 能够保持对会话密钥的保密性, 并对所发送消息添加了签名认证机制, 因此能有效抵御中间人攻击。

3.3 传输层安全协议标准

传输层安全 (transport layer security, TLS) 协议用于在两个通信应用之间提供保密性和认证性, 协议本身由 TLS 握手协议 (TLS handshake) 和 TLS 记录协议 (TLS record) 组成, 其中 TLS 握手协议负责服务器向客户端认证自身进而与客户端协商密钥; TLS 记录协议使用握手协议协商的密钥对消息数据进行加密与认证。TLS1.3 协议采用基于椭圆曲线上的 Diffie-Hellman 密钥协商协议, 其基本过程如下:

1. 客户端随机选择 $r_C = \{0, 1\}^{256}$, $x \in \mathbb{Z}_q$, 计算 $X = g^x$, 将 $\{r_C, X\}$ 发送给服务器。
2. 服务器端 S 随机选择 $r_S = \{0, 1\}^{256}$, $y \in \mathbb{Z}_q$, 计算 $Y = g^y$, S 计算 $Z = X^y$, 并进一步得到以下密钥

$$\begin{cases} hts_C || hts_S = F_1(Z, r_C || X || r_S || Y), \\ htk_C || htk_S = KDF(hts_C || hts_S, \epsilon) \end{cases}$$

其中 ϵ 为固定标签信息, hts_C, hts_S 分别是用以生成客户端与服务器, 在握手阶段的密钥, 保护握手阶段需要保密的信息。服务器生成签名:

$$\begin{cases} H_1 = H(r_C || \dots || Cert_S) \\ \sigma_S = Sign_{sk_S}(l_{SCV} || H_1) \end{cases}$$

其中 l_{SCV} 是固定的标签信息, H_1 表示服务器当前接收与本步将要发送的所有消息杂凑值. 计算:

$$\begin{cases} fk_S = HKDF(hts_S, l_5, \mu) \\ fk_C = HKDF(hts_C, l_5, \mu) \\ H_2 = H(r_C || \dots || Cert_S || \sigma_S) \\ fin_S = HMAC(fk_S, H_2) \end{cases}$$

其中 l_5 表示固定的标签信息, μ 表示 HKDF 的输出长度. 使用 htk_S 加密 $Cert_S || \sigma_S || fin_S$, 将 $\{r_S, Y, Enc_{htk_S}(Cert_S || \sigma_S || fin_S)\}$ 发送给客户端.

3. 客户端收到信息后, 计算 $Z = Y^x$, 同 2 步计算方法, 得到 $hts_C || hts_S$ 与 $htk_C || htk_S$, 使用 htk_S 解密信息, 验证 $Cert_S, \sigma_S, fin_S$ 是否合法. 计算

$$\begin{cases} H_3 = H(r_C || \dots || Cert_S || Cert_C) \\ \sigma_C = Sign_{sk_C}(l_{CCV} || H_3) \\ H_4 = H(r_C || \dots || Cert_S || \sigma_S || Cert_C || \sigma_C) \\ fin_C = HMAC(fk_C, H_4) \end{cases}$$

其中 l_{CCV} 表示固定标签信息. 使用 htk_C 加密 $\{Cert_C || \sigma_C || fin_C\}$, 将密文 $Enc_{htk_C}(Cert_C || \sigma_C || fin_C)$ 发送给服务器.

4. 服务器使用 htk_C 解密密文, 分别验证 $Cert_C, \sigma_C, fin_C$ 是否合法, 计算以下密钥

$$\begin{cases} ats_C || ats_S = F_2(Z, r_C || \dots || fin_S) \\ atk_C || atk_S = KDF(ats_C || ats_S, \epsilon) \\ ems = F_3(Z, r_C || \dots || fin_S) \\ rms = F_3(Z, r_C || \dots || fin_C) \end{cases}$$

其中 atk_C 与 atk_S 分别表示客户端与服务器的应用流量密钥, 用于记录层的认证加密. 同理客户端也计算得到上述密钥.

4 项目架构

4.1 安全目标

根据实验指导书所述场景, 可以明确该 AI 智能体协作安全协议应当满足如下安全目标 (包括实验要求和额外添加):

1. 可验证身份绑定: 所有智能体持有 CA 颁发的证书, 接收方通过 CA 公钥验证证书真伪, 从而验证对方身份, 杜绝伪造的智能体接入交互环节.
2. 数据传输机密性: 涉及用户隐私的数据, 包括行程信息、订单价格、授权凭证数据等均需要保密, 杜绝数据泄露风险.
3. 授权不扩散: Alice 生成的授权凭证包括: 总预算、授权权限、授权时效、被授权智能体 ID, 完整内容由 Alice 签名认证. 凭证内容一旦被篡改, 任务随即自动终止, 杜绝授权扩散、越权操作. 同时授权凭证仅单次会话有效, 不可复用.

4. 执行全程可追溯:FlightAgent 完成业务执行后, 必须生成带自身私钥签名的审计回执, 回执包含: 会话 ID、执行时间、订单详情、实际花费、授权凭证摘要、执行智能体 ID。Alice 汇总所有回执后, 可通过对应智能体公钥验证回执真伪, 所有交互日志、凭证、回执可本地留存, 支持事后全流程审计追溯, 不可抵赖。
5. 完整性、时效性与防重放: 协议中所有关键消息都必须能够验证是最新的、只用一次的、未被篡改的, 即效信息内容的完整性校验、防重放、序列号/时间戳验证、会话绑定。

4.2 流程设计

根据上述安全要求, 我们设计的 AI 智能体协作航班预定场景协议流程如下:

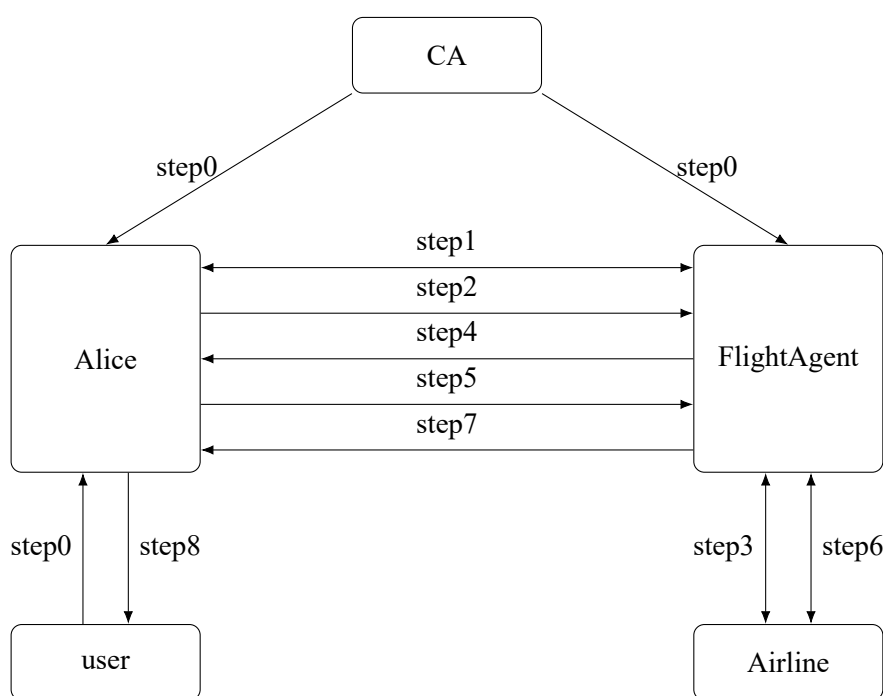


Figure 1: Multi-Agent Flight Booking Protocol

其中 CA 表示证书认证机构,Alice 为用户智能体,FlightAgent 为航班预定智能体,user 为用户,Airline 为航空公司 (也可以是智能体)。上述协议具体步骤如下:

- step0 Alice 和 FlightAgent 在 CA 处注册得到各自的证书 certificate,user 向 Alice 发出预定机票的命令:” 预订下周 (2026.06.08-2026.06.16) 青岛—纽约往返机票, 总预算不超过 10000 元, 单人出行”, 此后仅需等待 Alice 与 FlightAgent 协作交互即可, 无需 user 介入操作。
- step1 Alice 与 FlightAgent 之间通过 TLS 协议验证对方身份并协商会话密钥。
- step2 Alice 生成授权凭证 AuthToken, 包括预算、行程、授权信息、授权权限、授权失效、被授权智能体 ID 等, 对这些信息计算杂凑值并签名, 将得到的 AuthorizationCredential 发送给 FlightAgent。

- step3 FlightAgent 接收到授权后先验证签名, 如果签名不通过则拒绝服务, 直接退出; 否则, 根据授权凭证的信息执行即机票查询操作, 将查询结果, 包括具体航班、日期、实际价格, 时间戳等信息一并计算杂凑值并签名, 将签名和加密后的查询信息发给 Alice 以询问是否购买预定。
- step4 Alice 收到 FlightAgent 发送来的信息后先验证签名, 如果签名不通过则拒绝服务并直接退出; 否则, 根据查询结果, 判断是否购买预定, 发送 BookRequest 给 FlightAgent 允许其预定机票。
- step5 FlightAgent 收到 Alice 发送来的 BookRequest 信息, 同样验证签名, 不通过则拒绝服务并退出, 否则执行 step6。
- step6 FlightAgent 在航空公司 Airline 处预定机票, 将具体订单 ID、航空公司信息、机票信息、具体花费等信息作为收据 Receipt, 签名并发回给 Alice。
- step7 Alice 验证签名, 保存审计记录, 并将结果报告给 user。
- step8 Alice 将最终结果报告给用户, 包括成功预定或中途发生的异常信息。